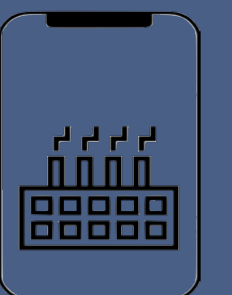
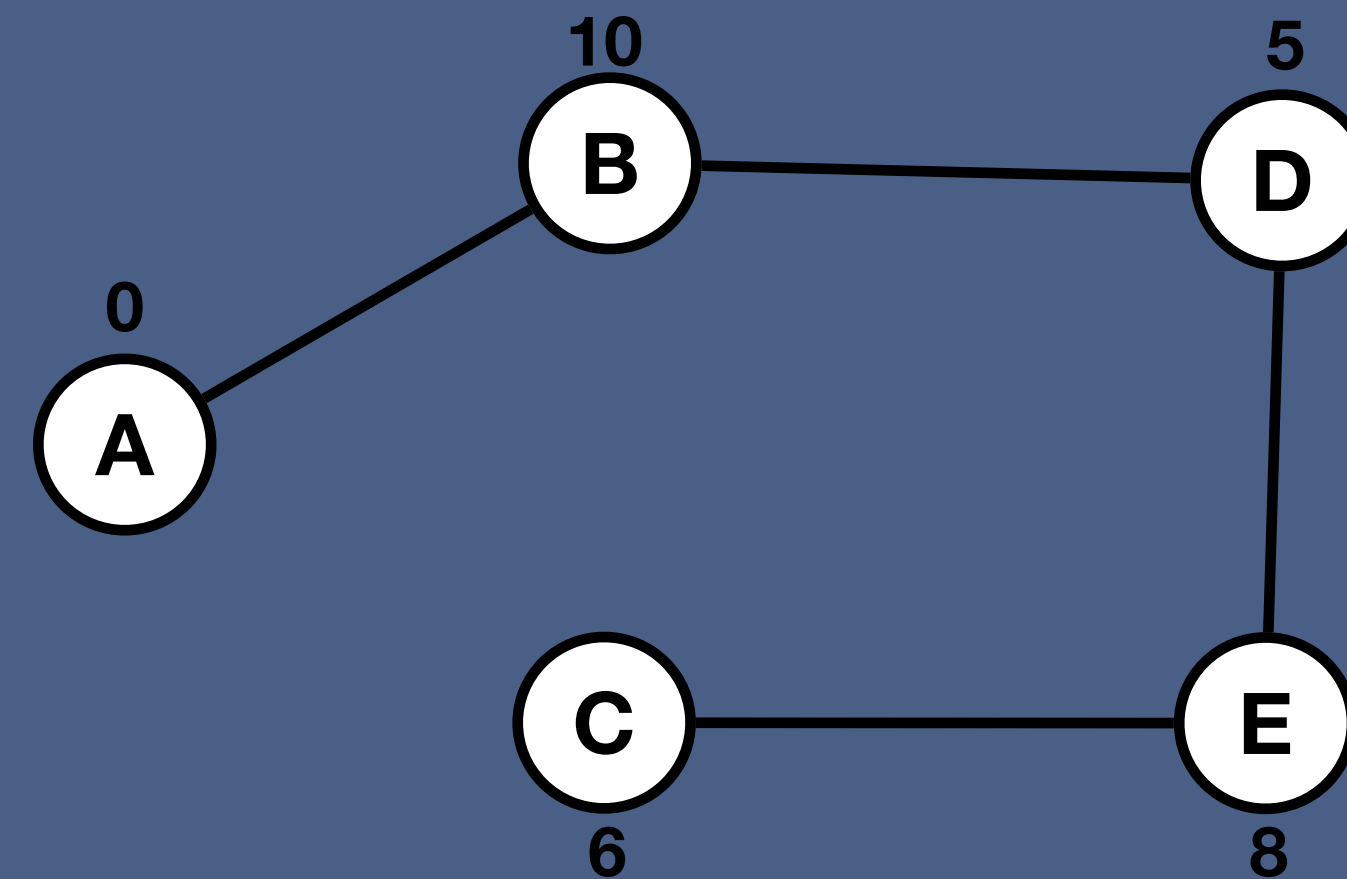
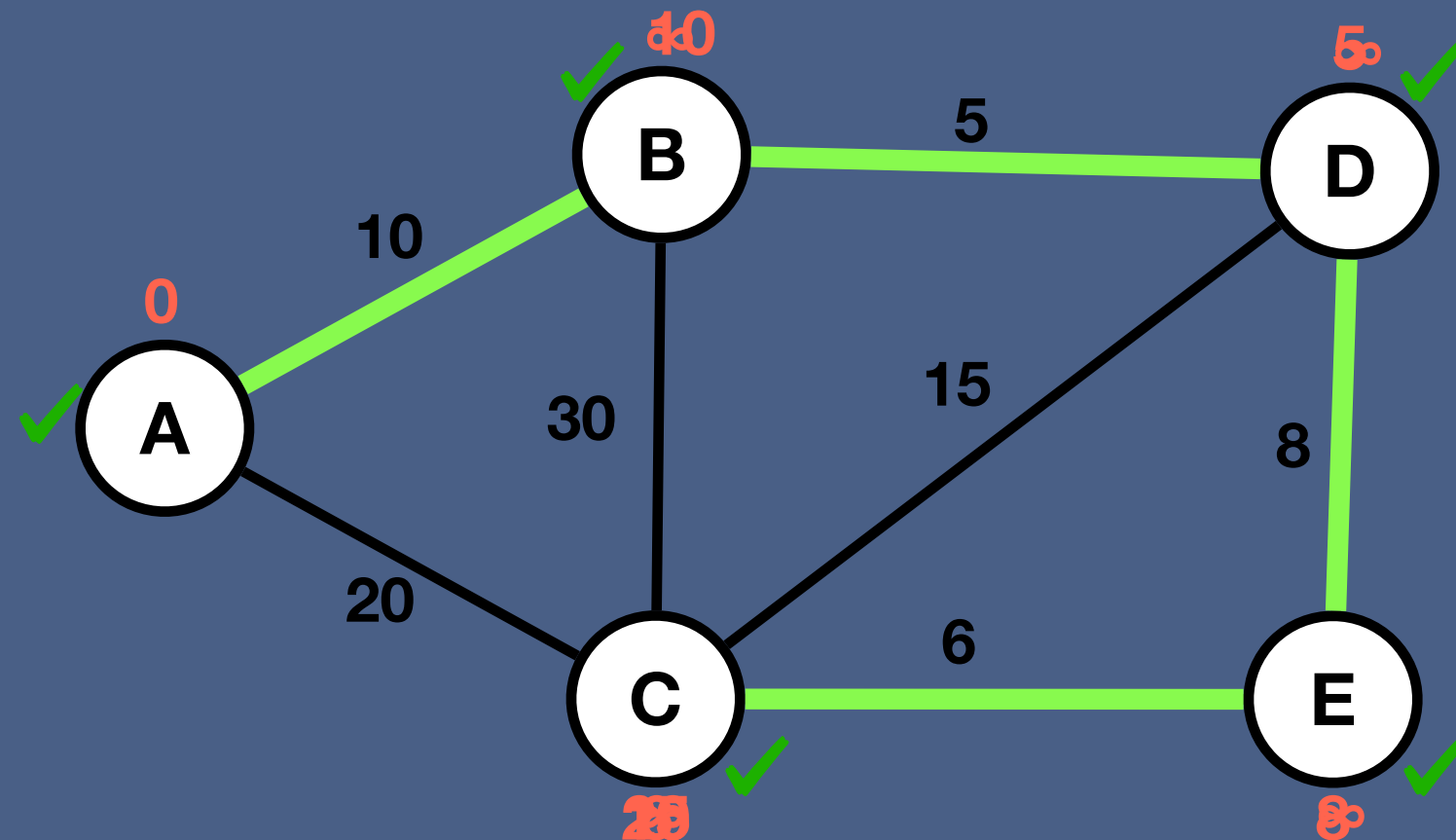


Prim's Algorithm

It is a greedy algorithm

It finds a minimum spanning tree for weighted undirected graphs in following ways

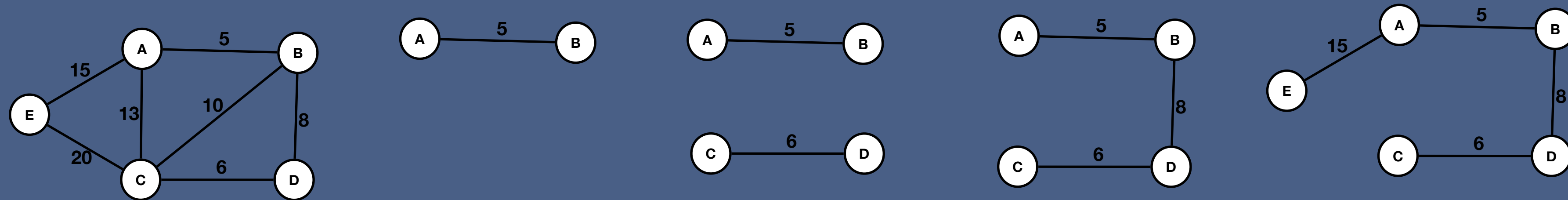
1. Take any vertex as a source set its weight to 0 and all other vertices' weight to infinity
2. For every adjacent vertices if the current weight is more than current edge then we set it to current edge
3. Then we mark current vertex as visited
4. Repeat these steps for all vertices in increasing order of weight



Kruskal vs Prim

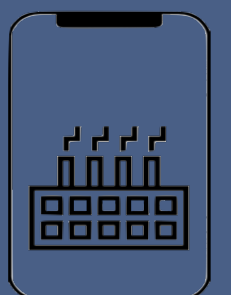
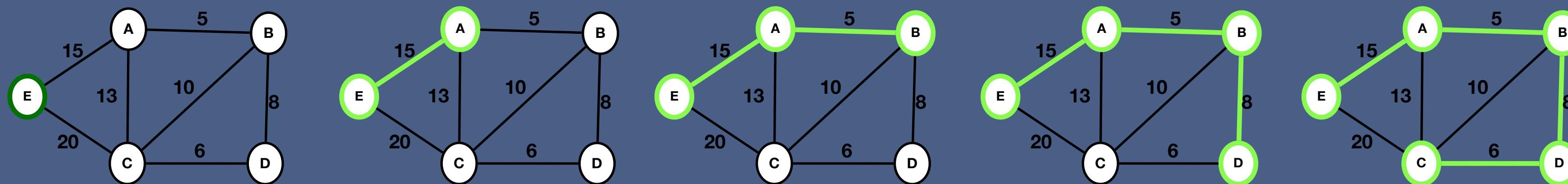
Kruskal

- Concentrates on Edges
- Finalize edge in each iteration



Prim's

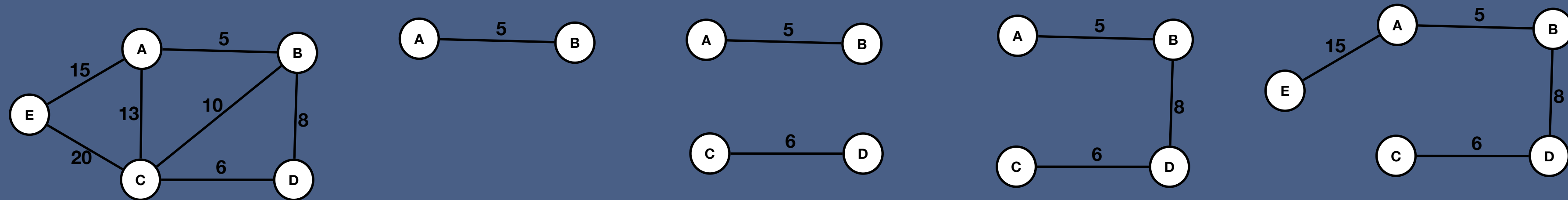
- Concentrates on Vertices
- Finalize Vertex in each iteration



Kruskal vs Prim

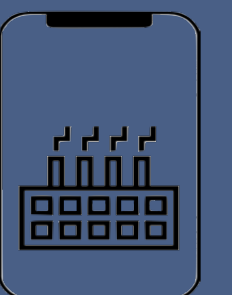
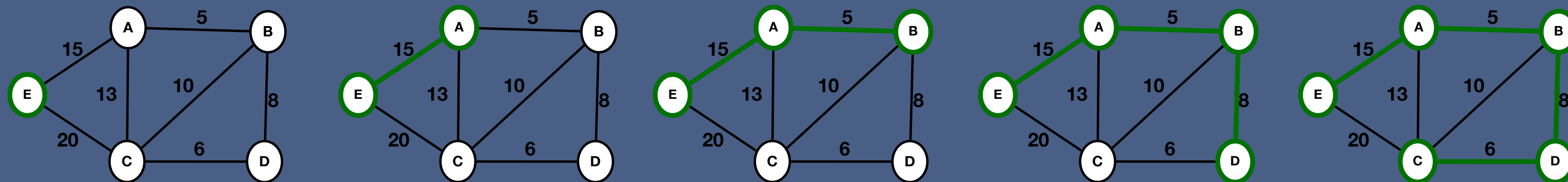
Kruskal

- Concentrates on Edges
- Finalize edge in each iteration



Prim's

- Concentrates on Vertices
- Finalize Vertex in each iteration



Kruskal vs Prim

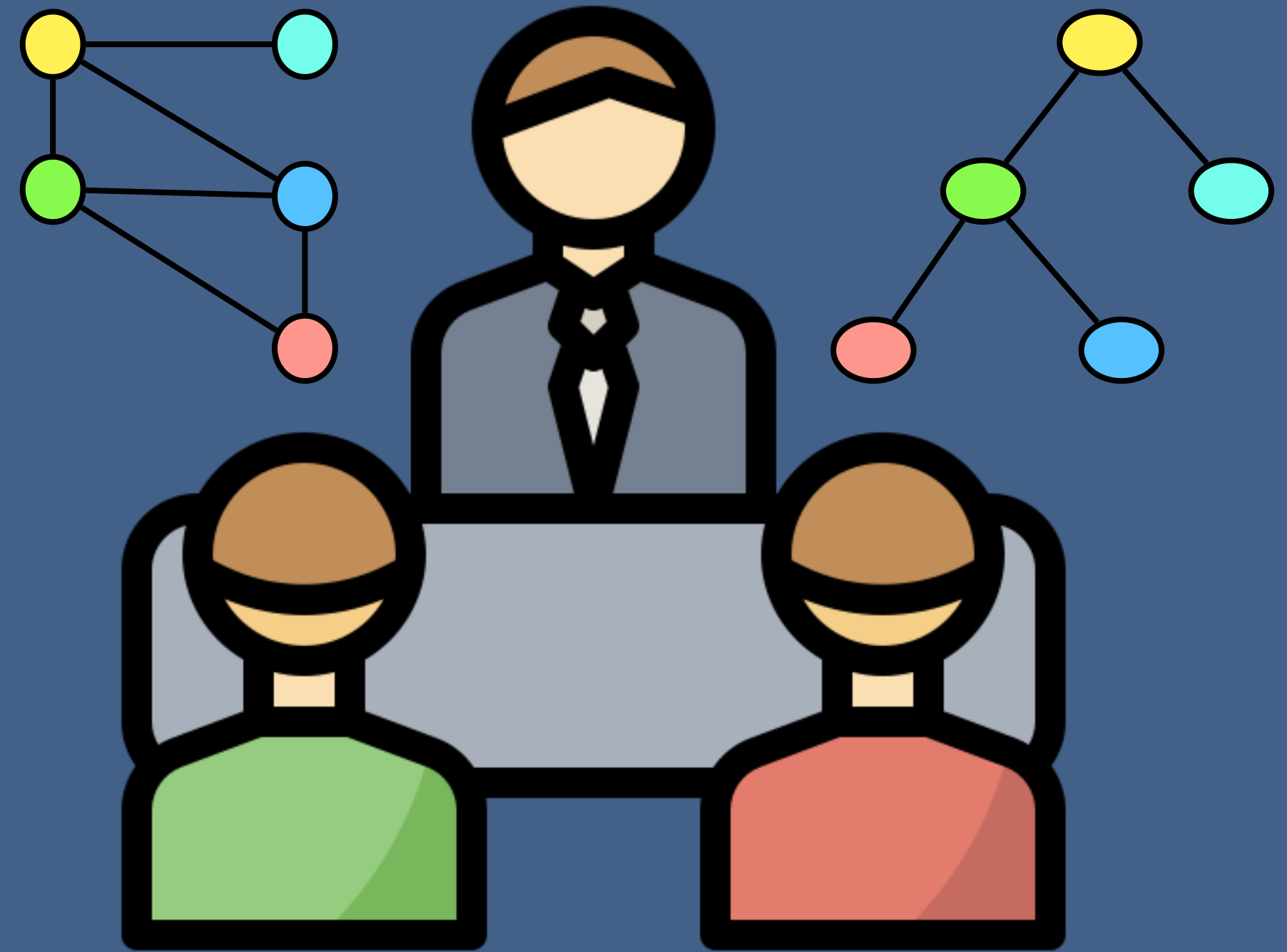
Kruskal Applications

- Landing cables
- TV Network
- Tour Operations
- LAN Networks
- A network of pipes for drinking water or natural gas.
- An electric grid
- Single-link Cluster

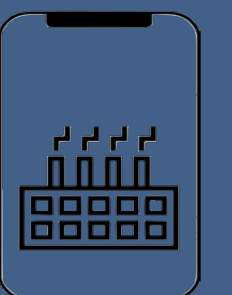
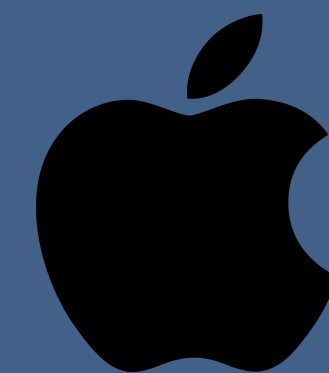
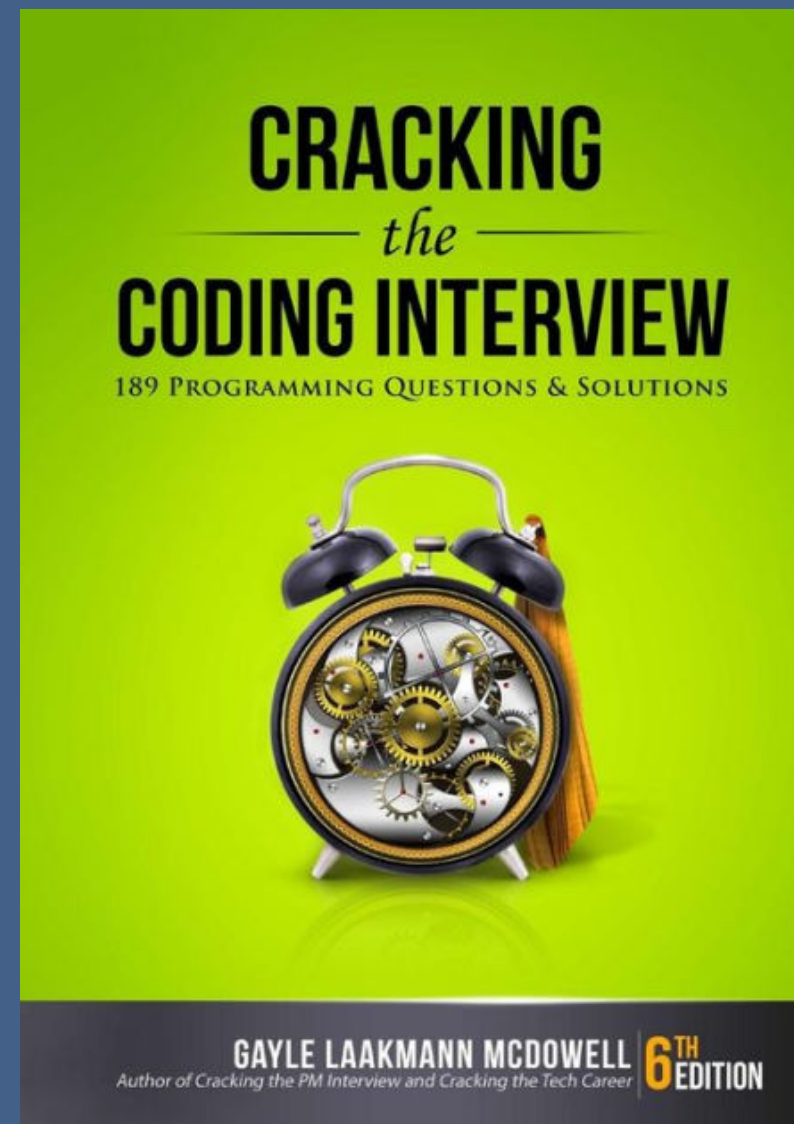
Prim's Applications

- Network for roads and Rail tracks connecting all the cities.
- Irrigation channels and placing microwave towers
- Designing a fiber-optic grid or ICs.
- Traveling Salesman Problem.
- Cluster analysis.
- Pathfinding algorithms used in AI(Artificial Intelligence).

Tree and Graph Interview Questions



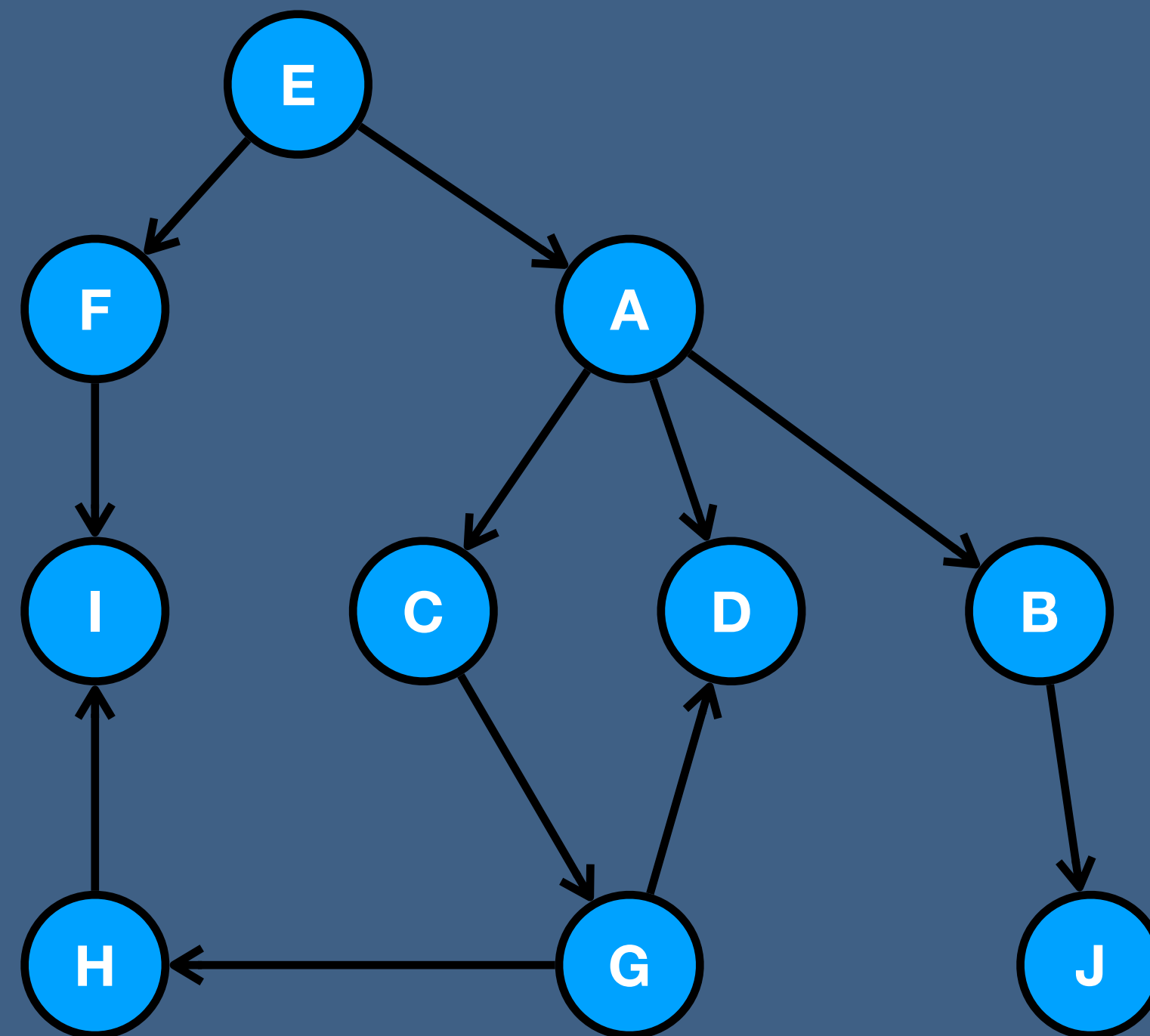
Tree and Graph Interview Questions



Route Between Nodes

Problem Statement:

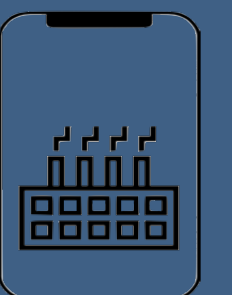
Given a directed graph and two nodes (S and E), design an algorithm to find out whether there is a route from S to E.



A $\rightarrow$ E False

A $\rightarrow$ J True

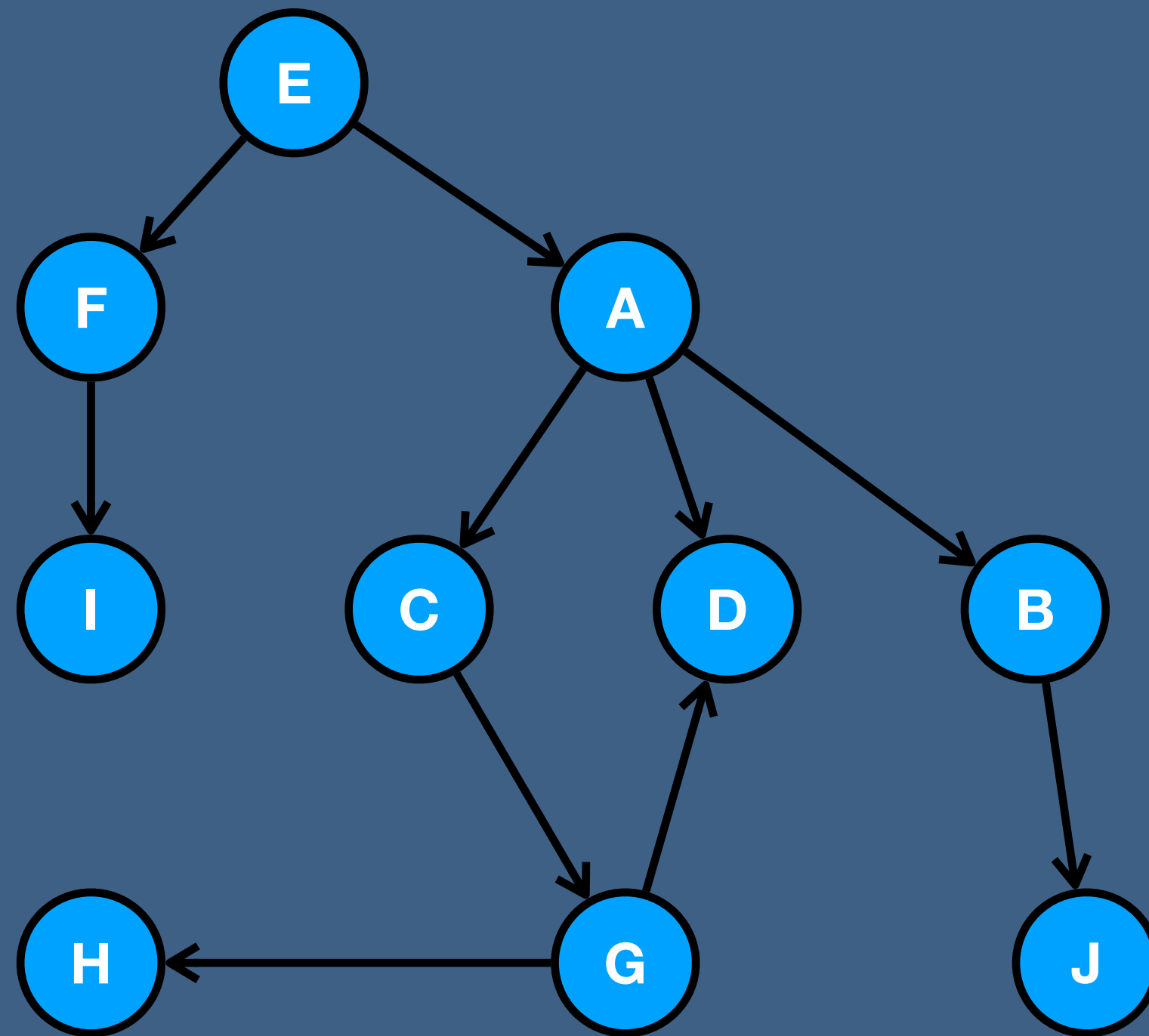
A $\rightarrow$ I True



Route Between Nodes

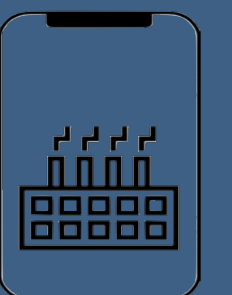
Problem Statement:

Given a directed graph and two nodes (S and E), design an algorithm to find out whether there is a route from S to E.



Pseudocode

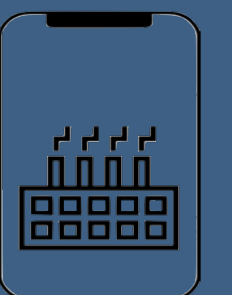
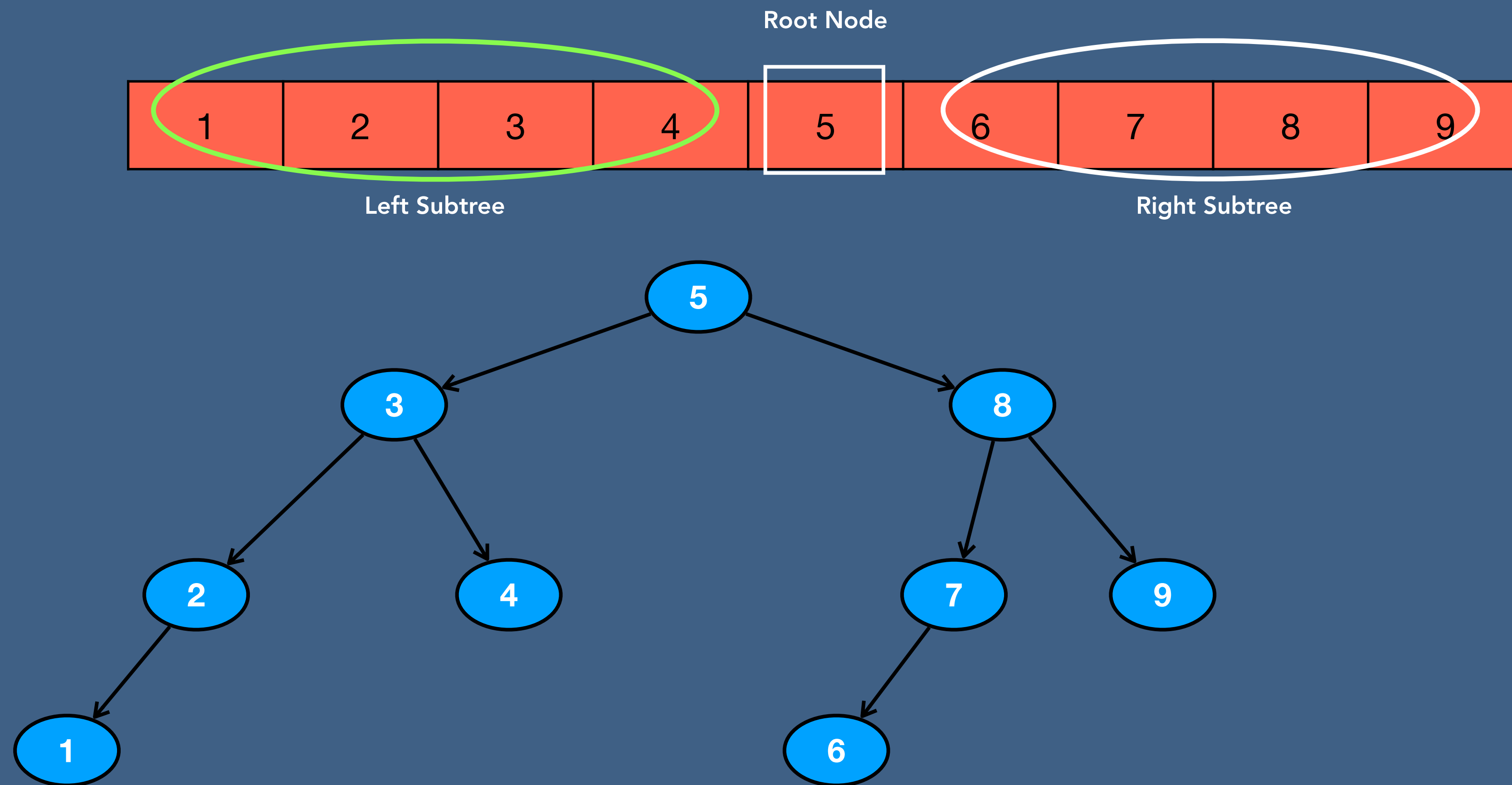
- Create function with two parameters start and end nodes
- Create queue and enqueue start node to it
- Find all the neighbors of the just enqueued node and enqueue them into the queue
- Repeat this process until the end of elements in graph
- If during the above process at some point in time we encounter the destination node, we return True.
- Mark visited nodes as visited



Minimal Tree

Problem Statement:

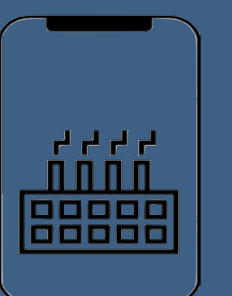
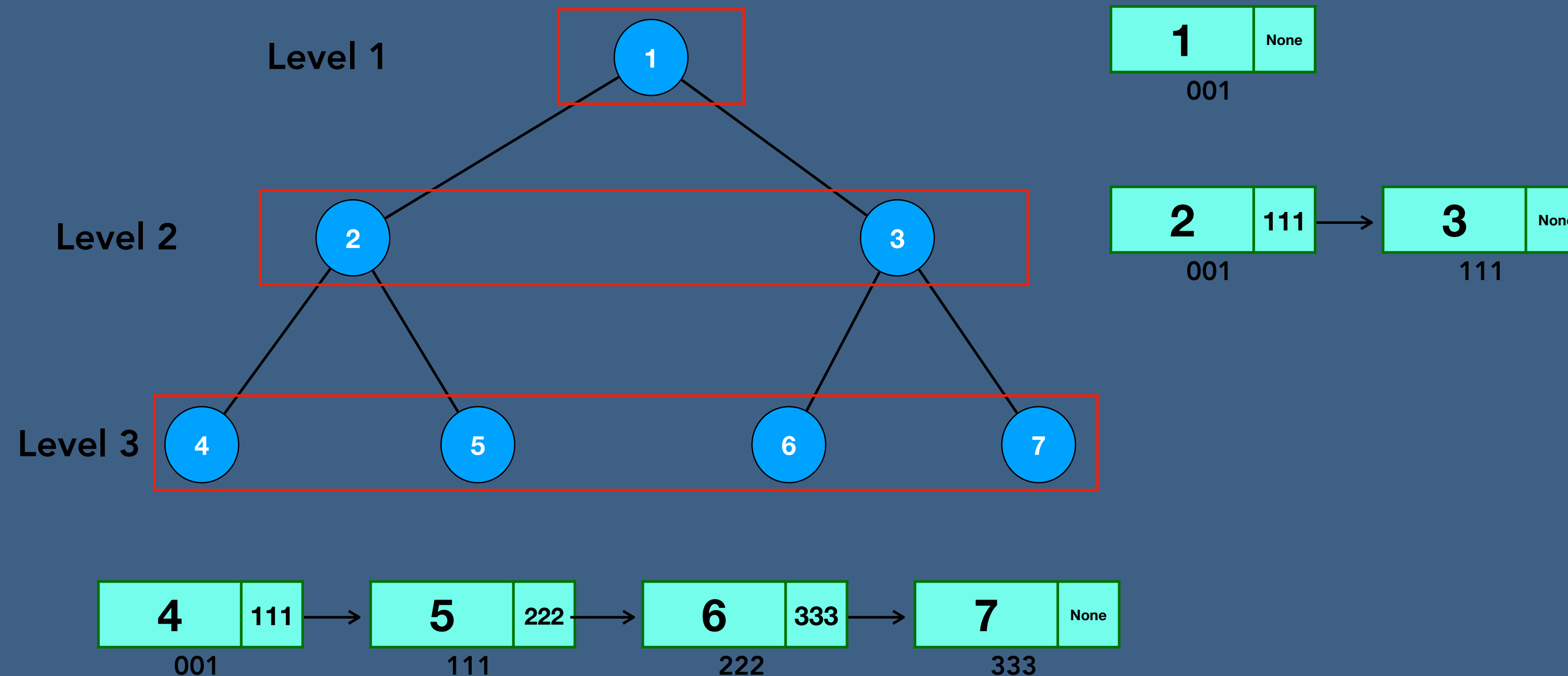
Given a sorted (increasing order) array with unique integer elements, write an algorithm to create a binary search tree with minimal height.



List of Depths

Problem Statement:

Given a binary search tree, design an algorithm which creates a linked list of all the nodes at each depth (i.e., if you have a tree with depth D, you'll have D linked lists)

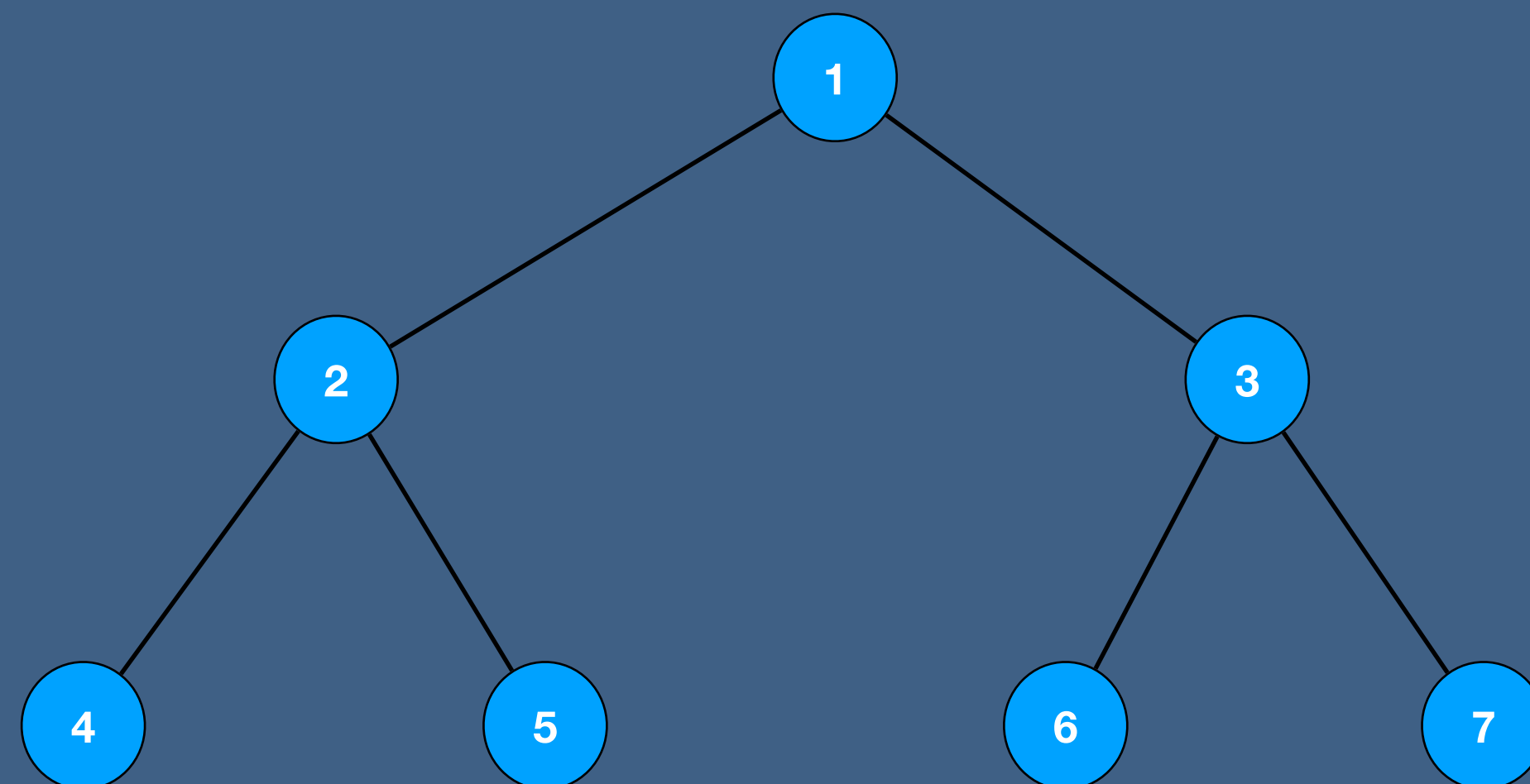


List of Depths

Problem Statement:

Given a binary search tree, design an algorithm which creates a linked list of all the nodes at each depth (i.e., if you have a tree with depth D, you'll have D linked lists)

Pre Order Traversal

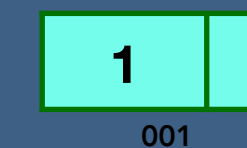


custDict

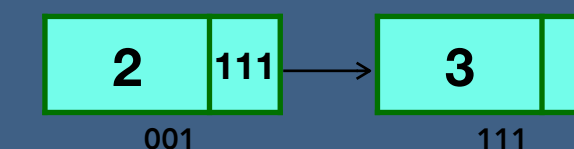
Keys

Values

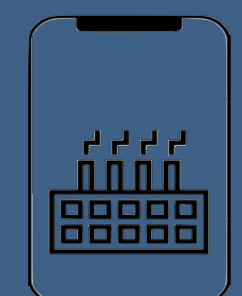
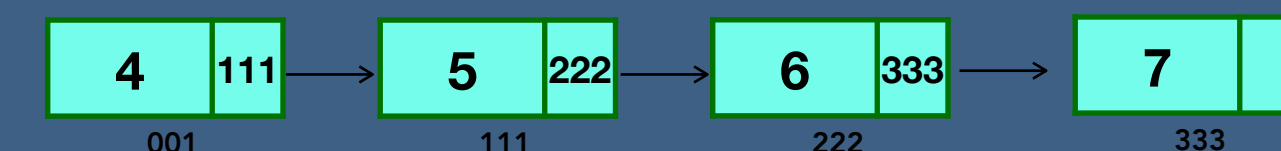
3



2



1

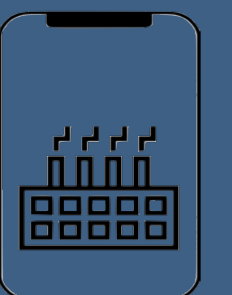
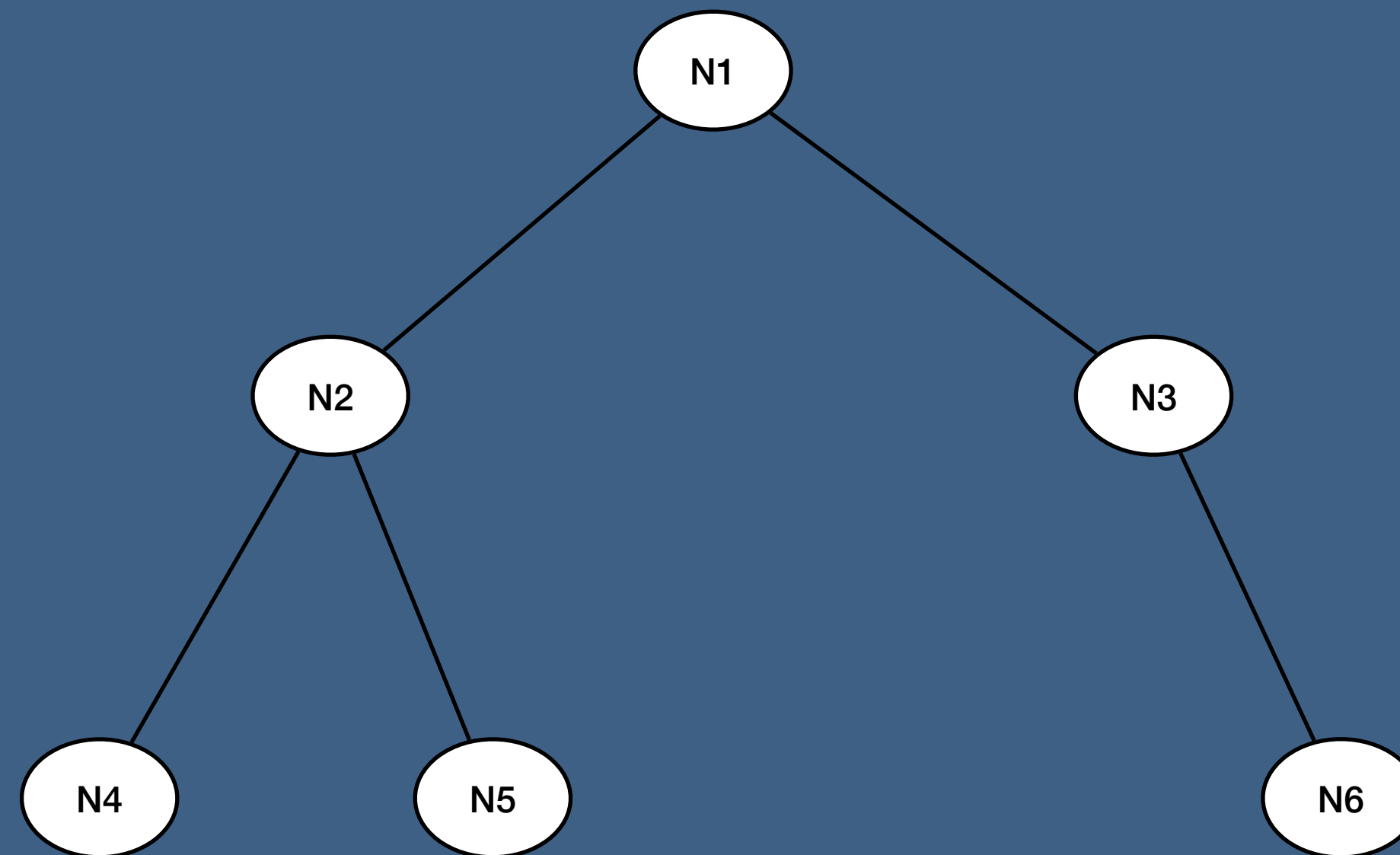


Check Balanced

Problem Statement:

Implement a function to check if a binary tree is balanced. For the purposes of this question, a balanced tree is defined to be a tree such that the heights of the two subtrees of any node never differ by more than one.

Balanced Binary Tree

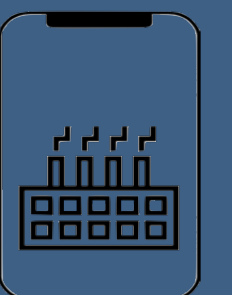
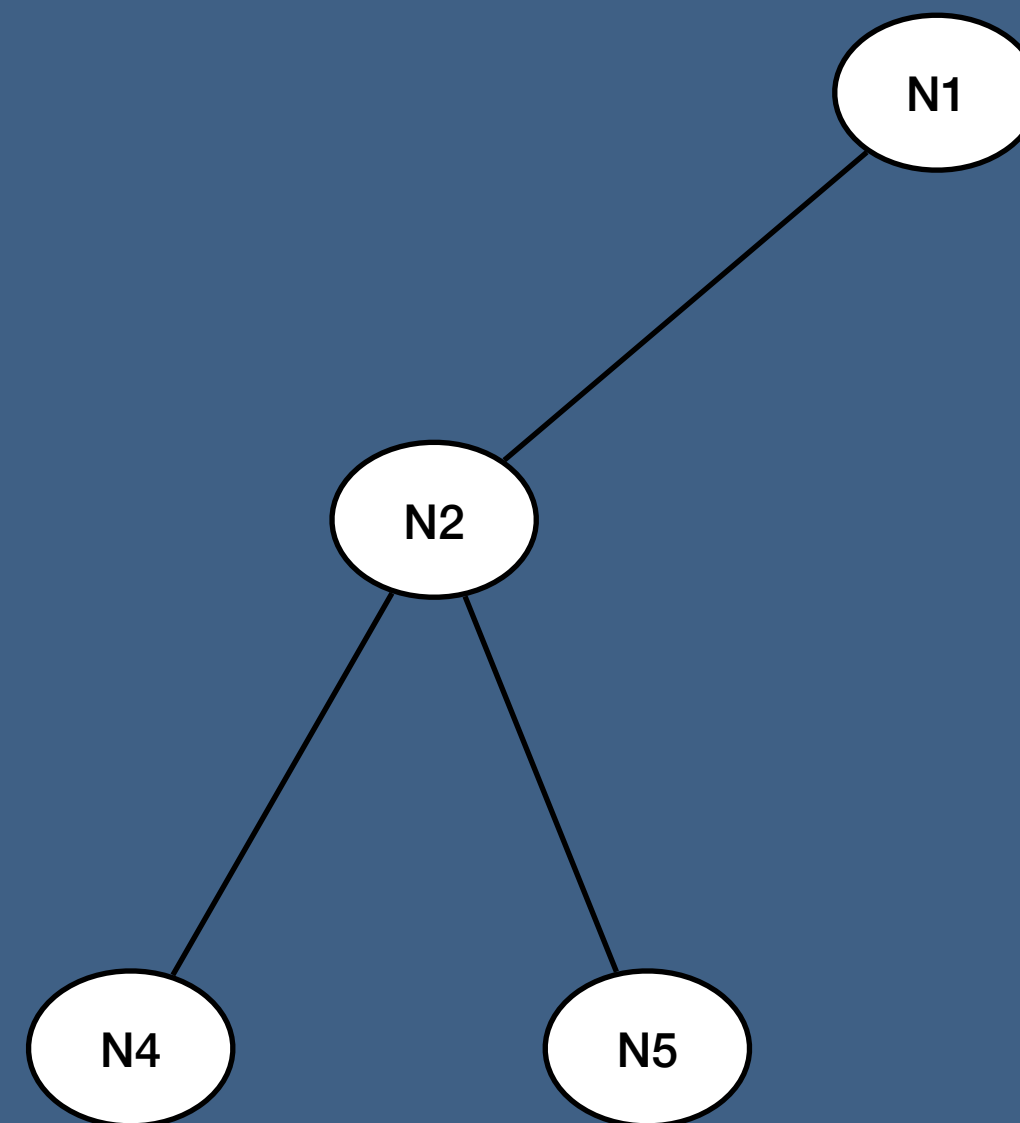


Check Balanced

Problem Statement:

Implement a function to check if a binary tree is balanced. For the purposes of this question, a balanced tree is defined to be a tree such that the heights of the two subtrees of any node never differ by more than one.

Balanced Binary Tree



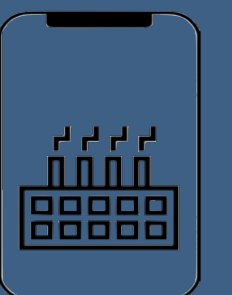
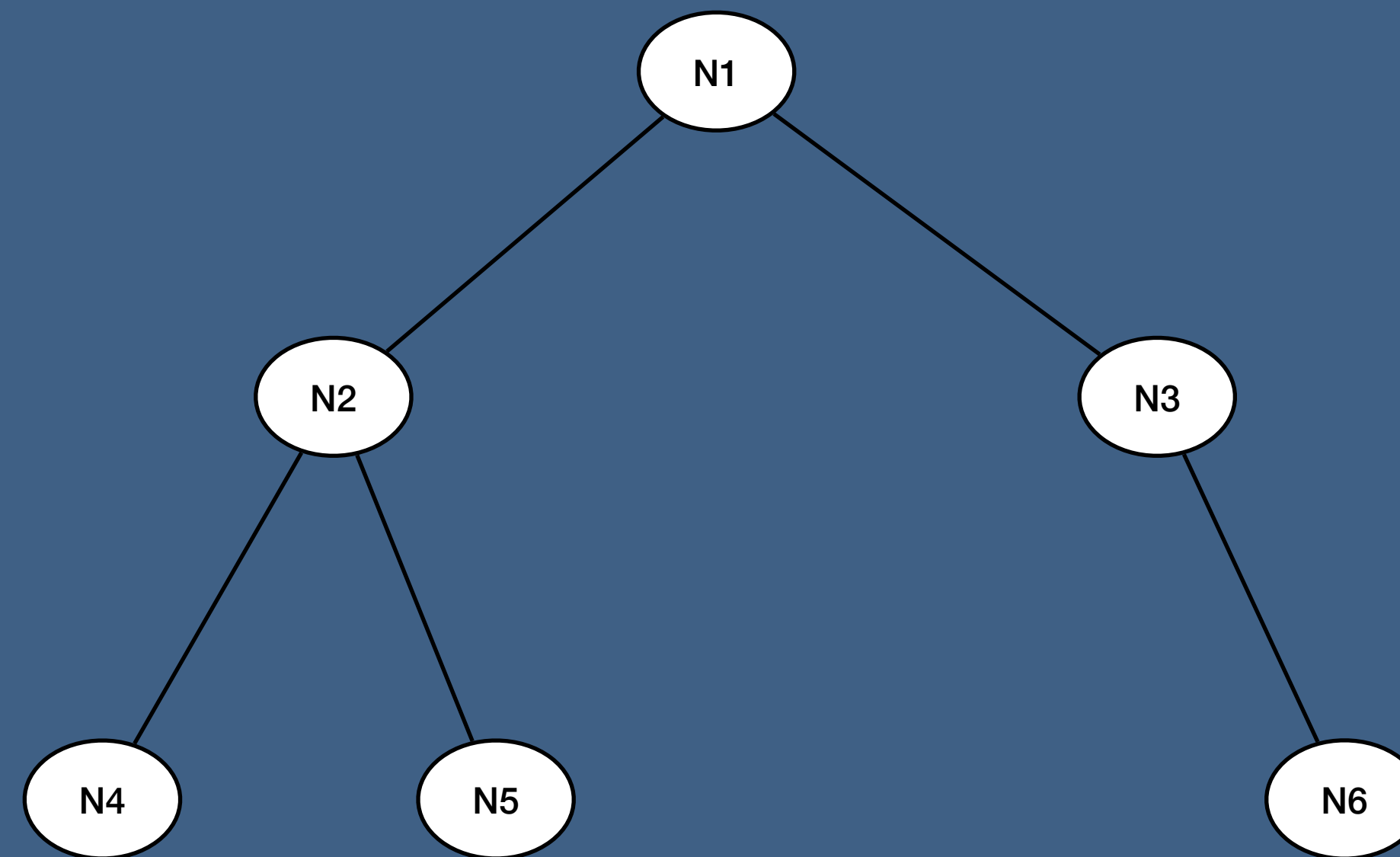
Check Balanced

Problem Statement:

Implement a function to check if a binary tree is balanced. For the purposes of this question, a balanced tree is defined to be a tree such that the heights of the two subtrees of any node never differ by more than one.

The Binary Tree is Balanced if:

- The right subtree is balanced
- The left subtree is balanced
- The difference between the height of the left subtree and the right subtree is at most 1



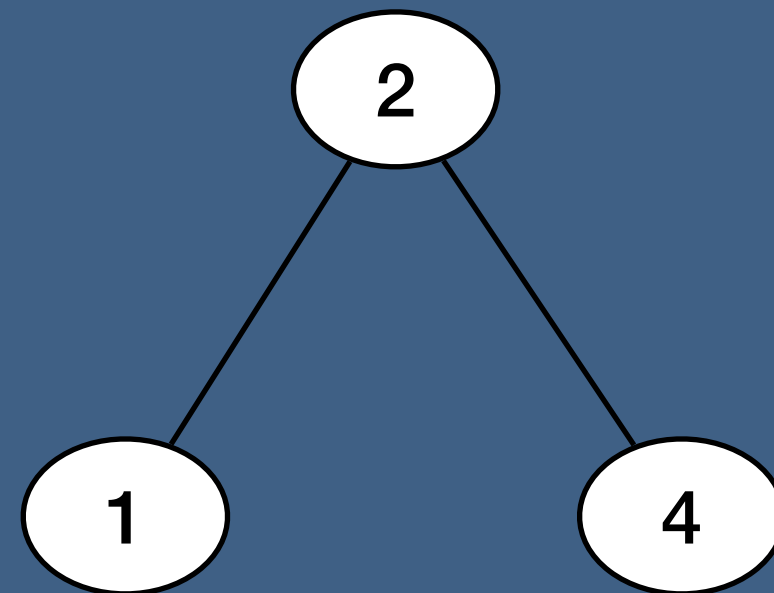
Validate BST

Problem Statement:

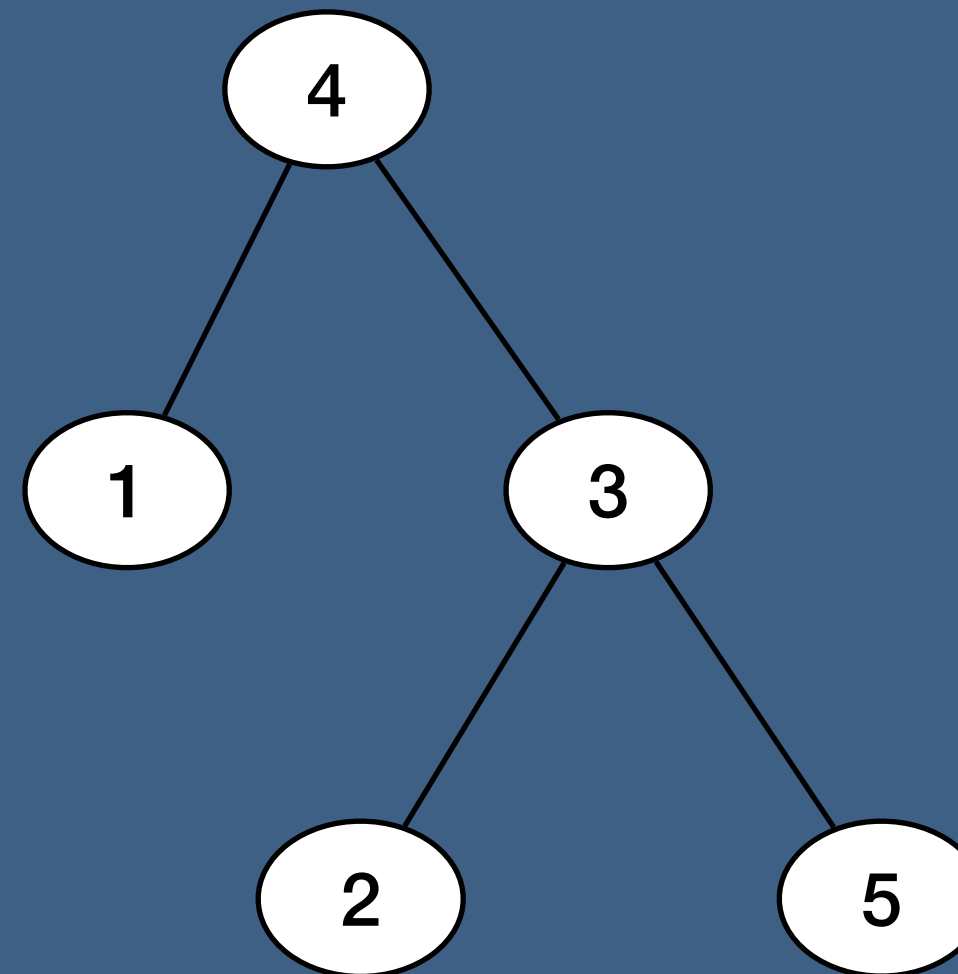
Implement a function to check if a binary tree is a Binary Search Tree.

The Binary Tree is Binary Search Tree if:

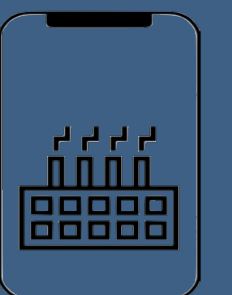
- The left subtree of a node contains only nodes with keys less than the node's key
- The right subtree of a node contains only nodes with keys greater than the node's key
- These conditions are applicable for both left and right subtrees



True



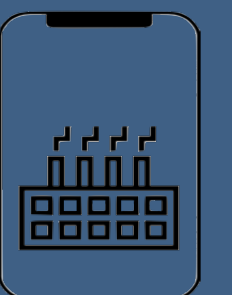
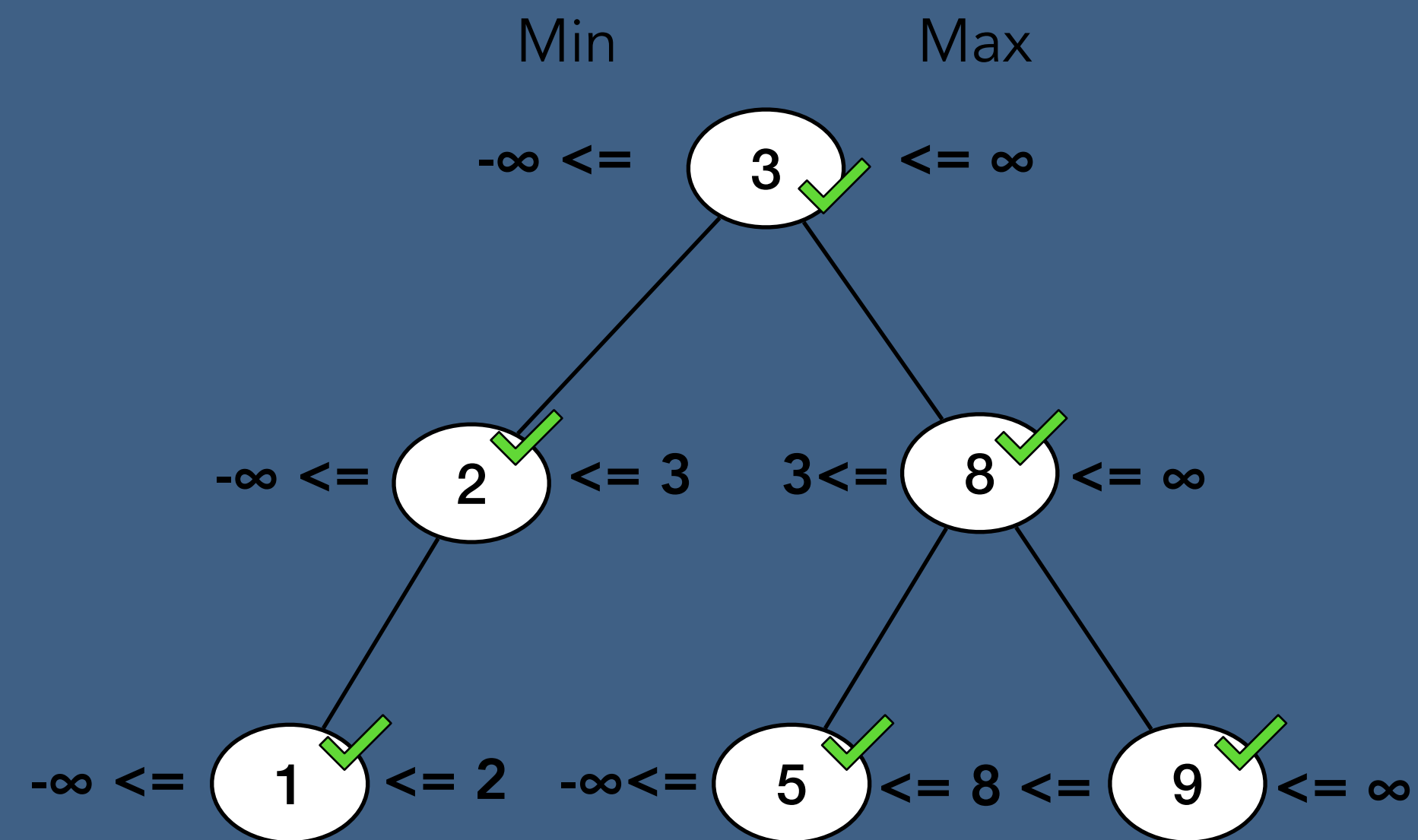
False



Validate BST

Problem Statement:

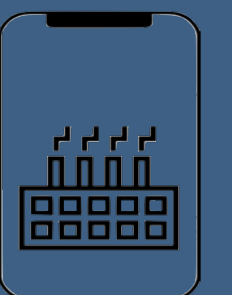
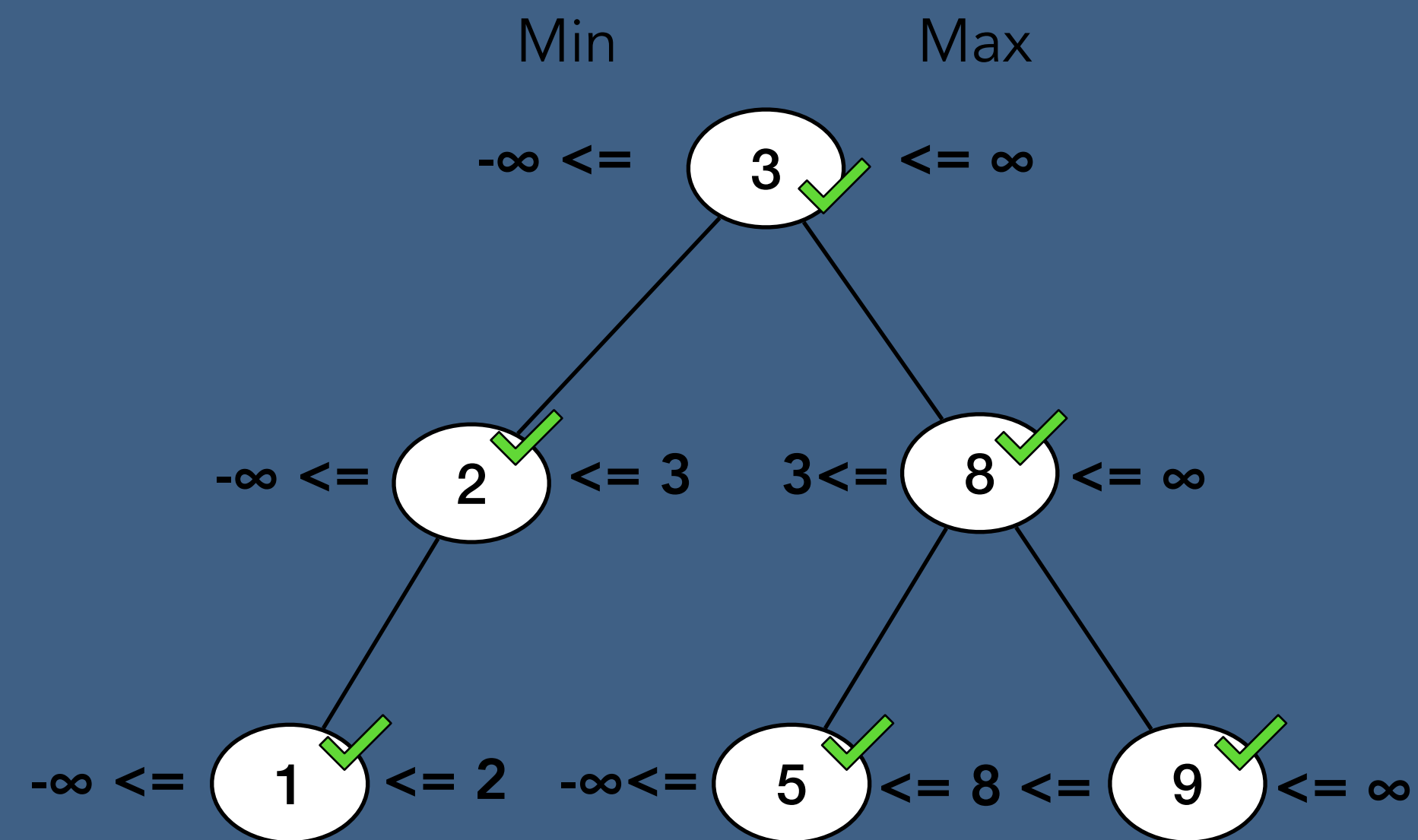
Implement a function to check if a binary tree is a Binary Search Tree.



Validate BST

Problem Statement:

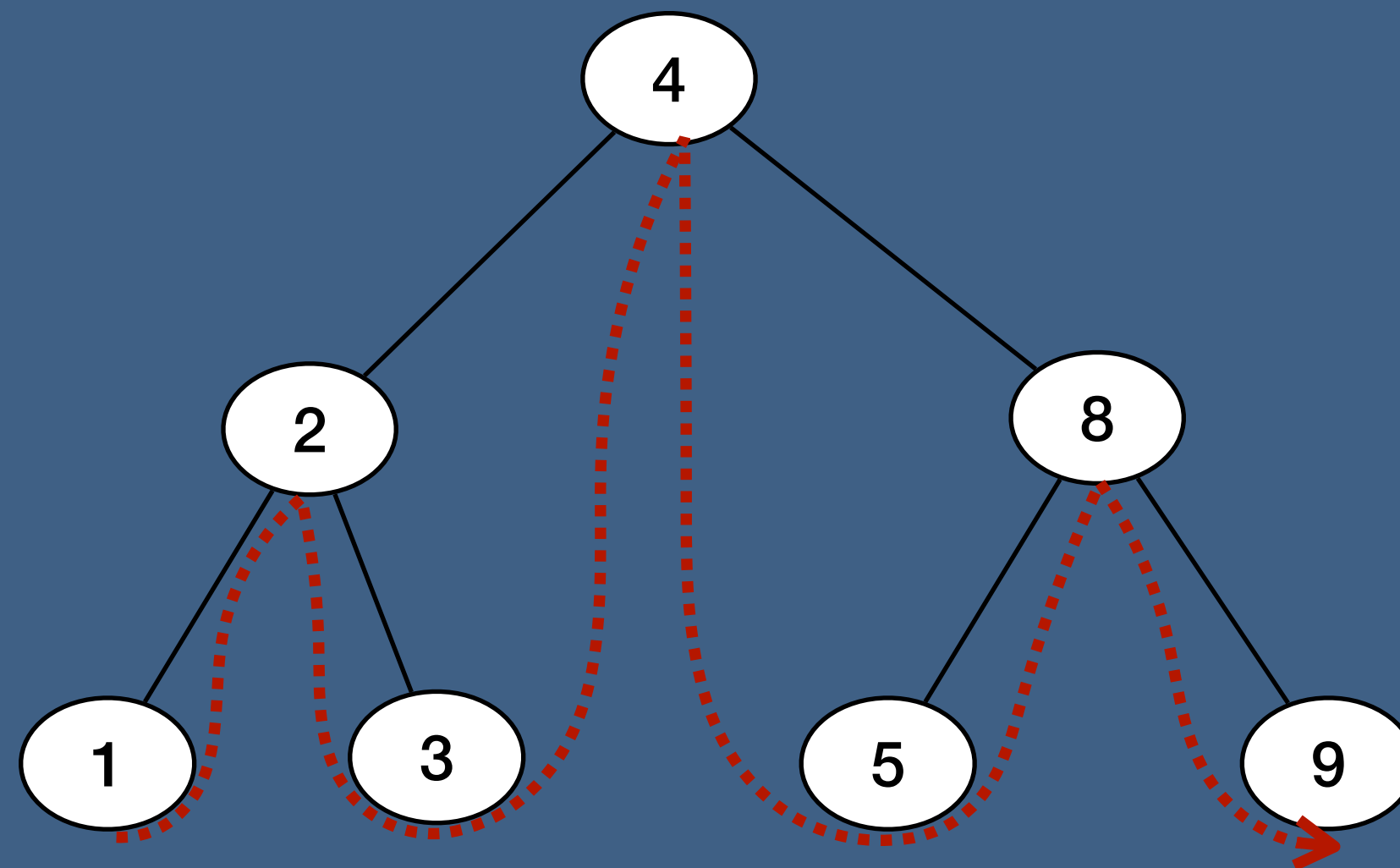
Implement a function to check if a binary tree is a Binary Search Tree.



Successor

Problem Statement:

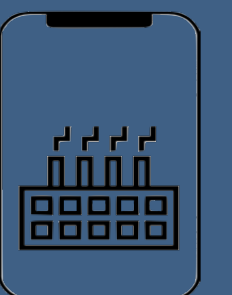
Write an algorithm to find the next node (i.e in-order successor) of given node in a binary search tree. You may assume that each node has a link to its parent.



In-order successor of **1** is **2**

In-order successor of **3** is **4**

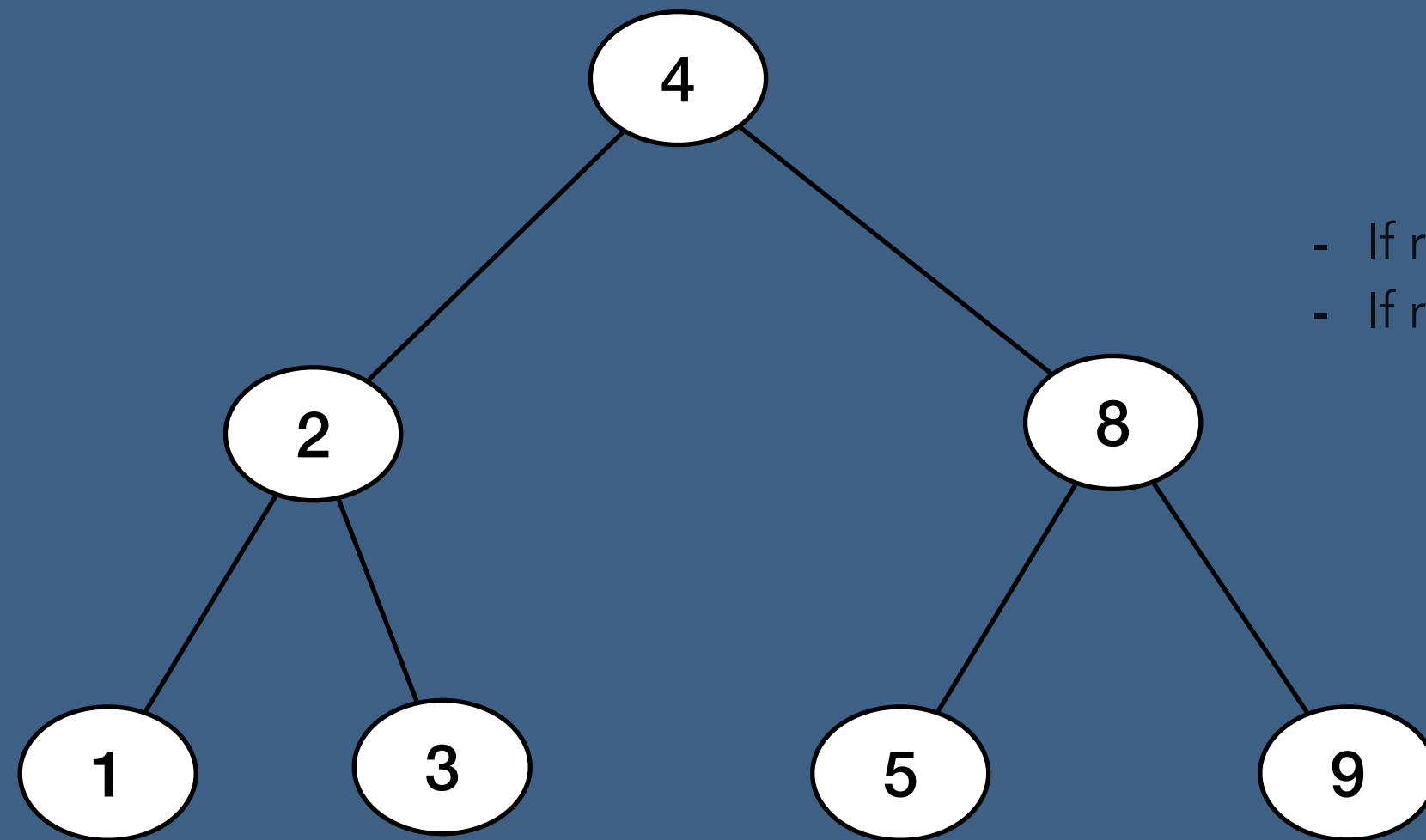
In-order successor of **2** is **3**



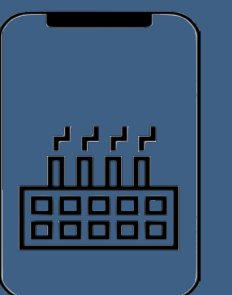
Successor

Problem Statement:

Write an algorithm to find the next node (i.e in-order successor) of given node in a binary search tree. You may assume that each node has a link to its parent.



- If right subtree of node is **not None**, then successor lies in right subtree.
- If right subtree of node is **None**, then successor is one of the ancestors.



Build Order

Problem Statement:

You are given a list of projects and a list of dependencies (which is a list of pairs of projects, where the second project is dependent on the first project). All of a project's dependencies must be built before the project is. Find a build order that will allow the projects to be built. If there is no valid build order, return an error.

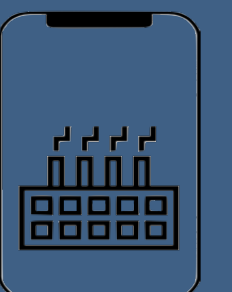
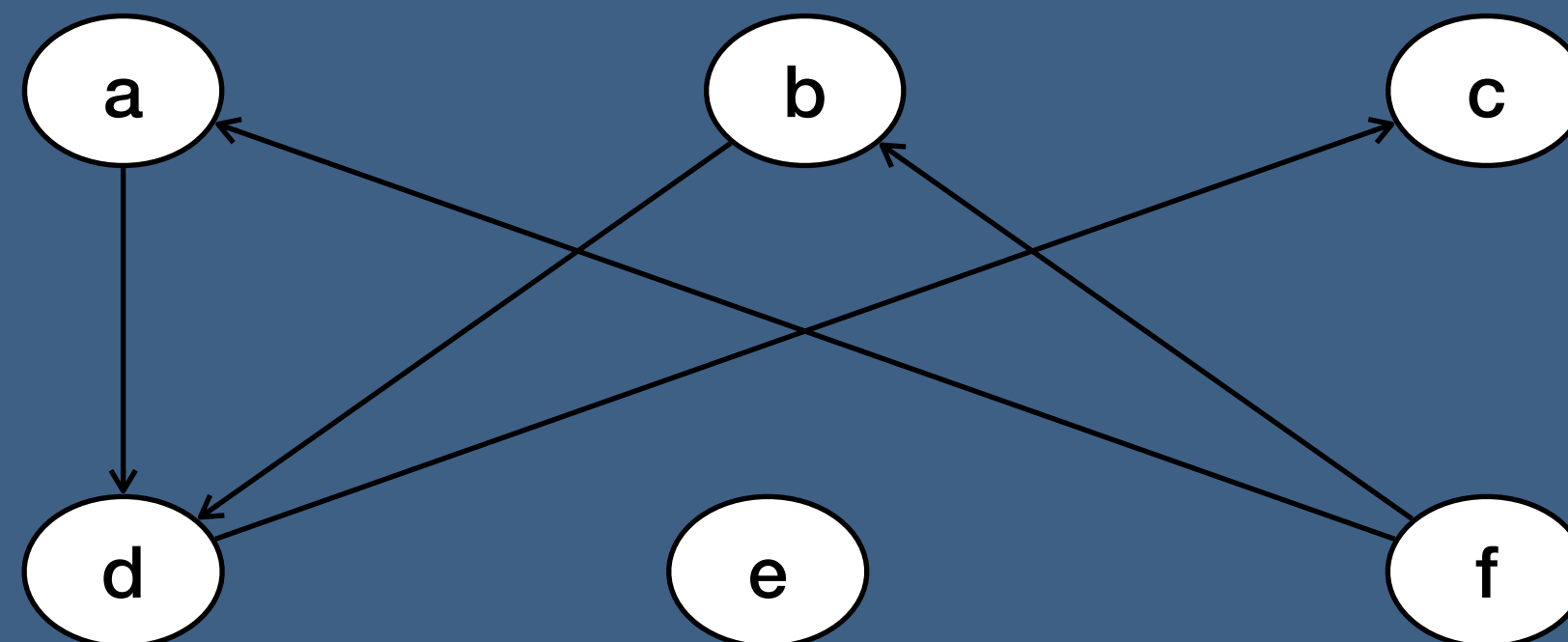
Input:

Projects : a, b, c, d, e, f

Dependencies: (a, d), (f, b), (b, d), (f, a), (d, c)

Output:

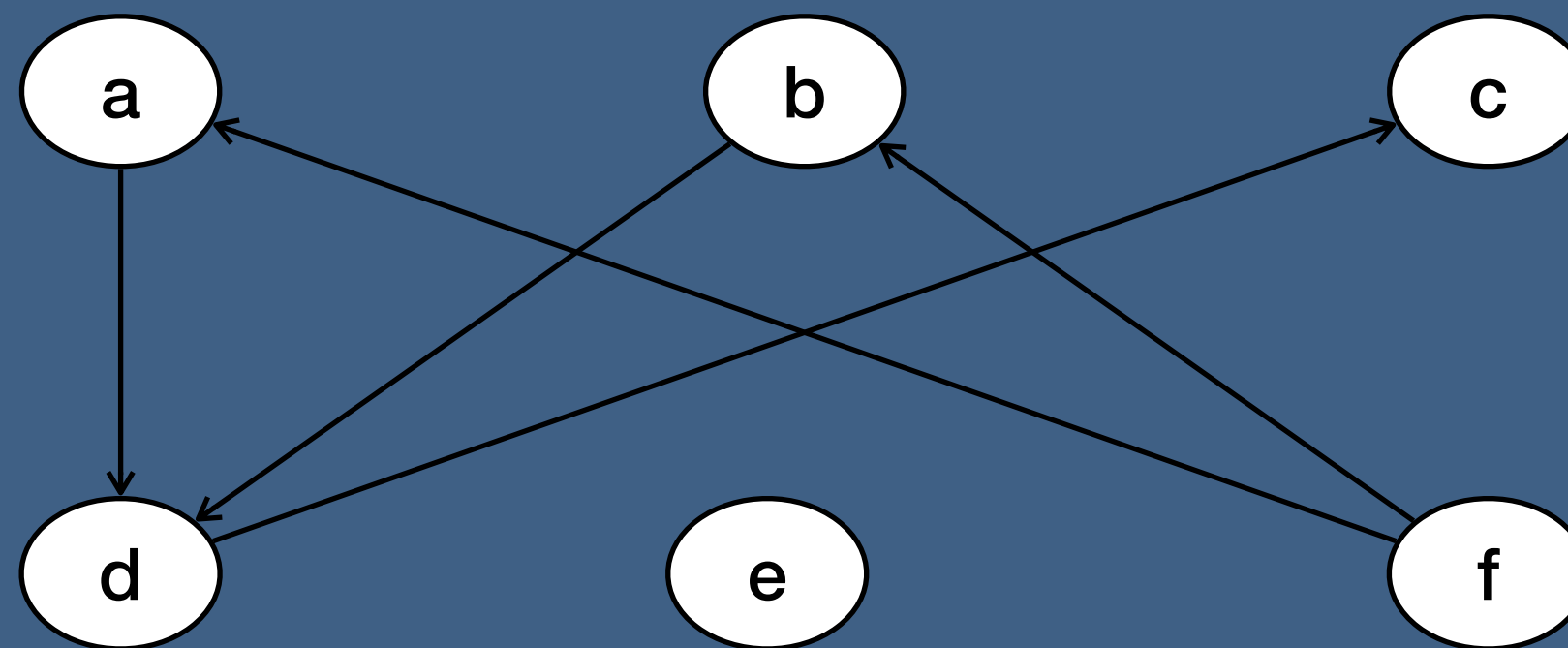
e, f, a, b, d, c



Build Order

Problem Statement:

You are given a list of projects and a list of dependencies (which is a list of pairs of projects, where the second project is dependent on the first project). All of a project's dependencies must be built before the project is. Find a build order that will allow the projects to be built. If there is no valid build order, return an error.



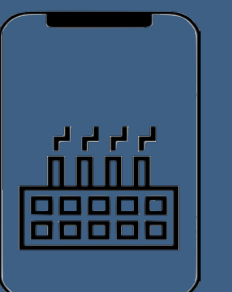
Find nodes with dependencies

a, b, c, d

Find nodes **without** dependencies

e, f

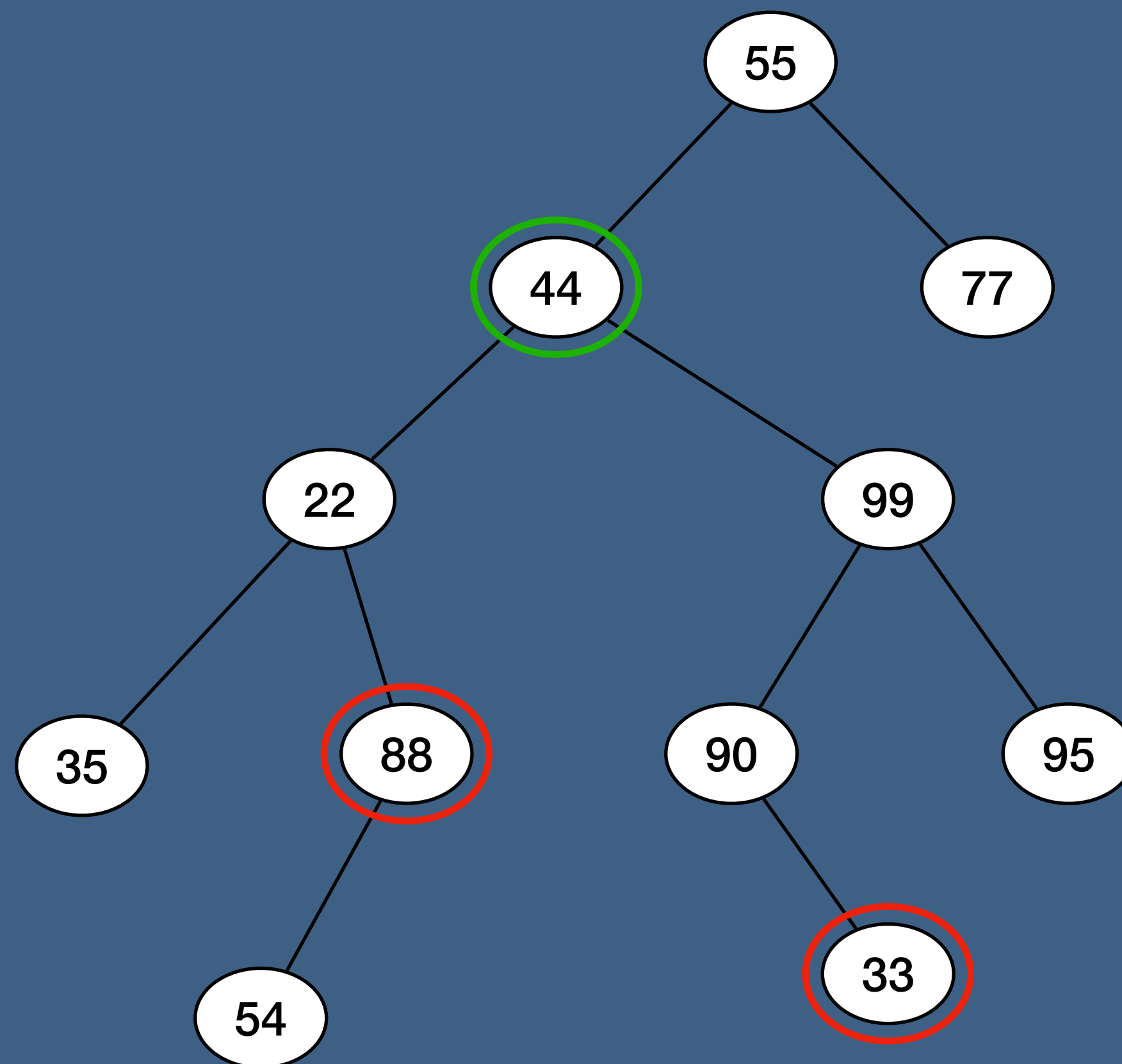
Find order by taking out nodes **without** dependencies.



First Common Ancestor

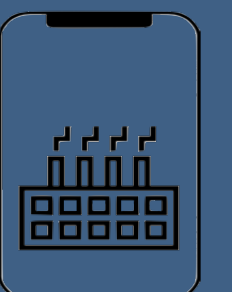
Problem Statement:

Design an algorithm and write code to find the first common ancestor of two nodes in a binary tree. Avoid storing additional nodes in a data structure. NOTE: This is not necessarily a binary search tree.



First Common Ancestor of 33 and 88

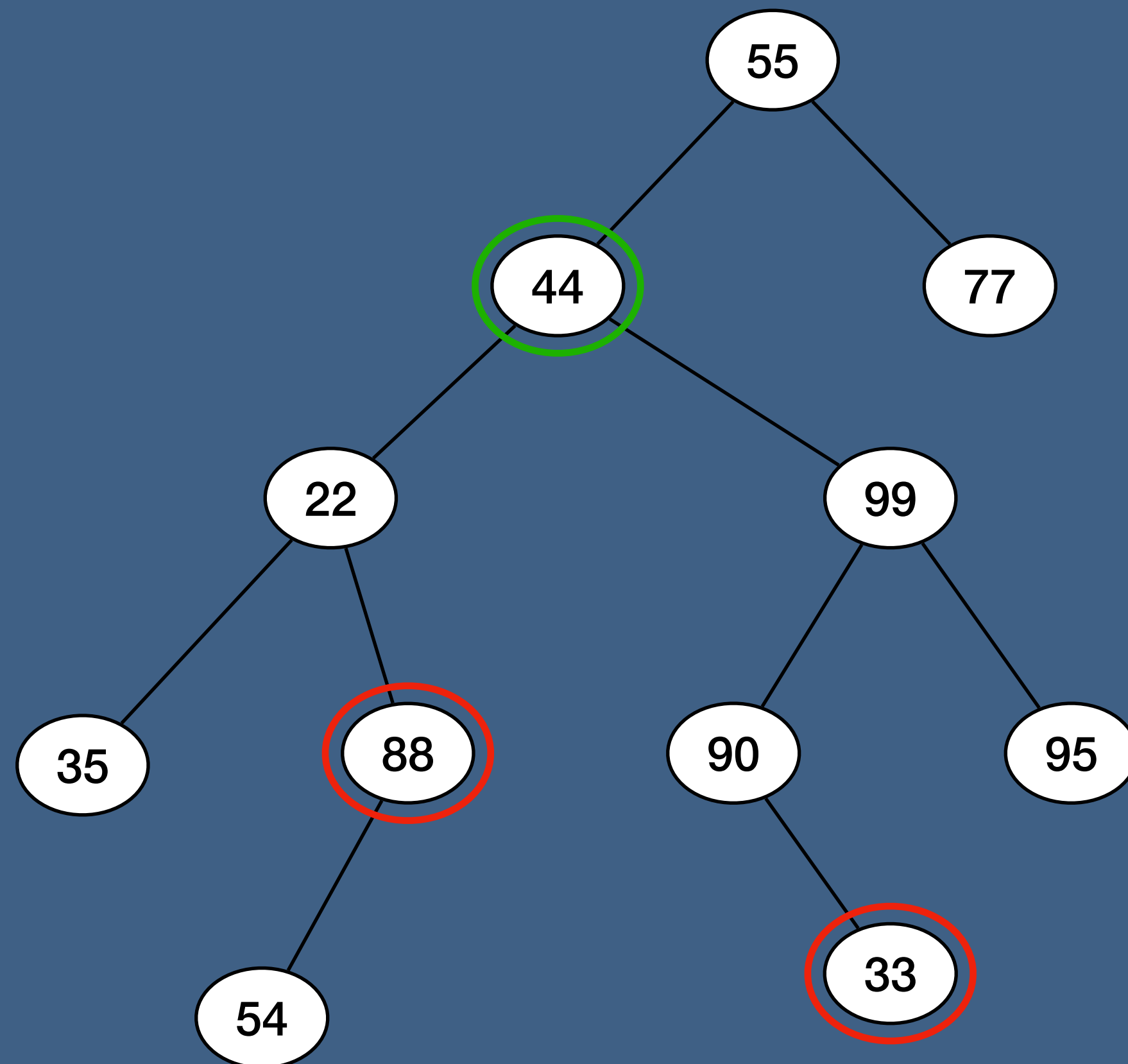
44



First Common Ancestor

Problem Statement:

Design an algorithm and write code to find the first common ancestor of two nodes in a binary tree. Avoid storing additional nodes in a data structure. NOTE: This is not necessarily a binary search tree.



findNodeinTree()

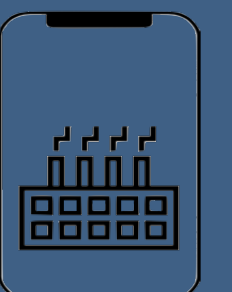
findNodeinTree(88, left-44) → True

findNodeinTree(33, left-44) → True

findNodeinTree(88, left-22) → True

findNodeinTree(33, left-22) → False

Return root (44)



First Common Ancestor

Problem Statement:

Design an algorithm and write code to find the first common ancestor of two nodes in a binary tree. Avoid storing additional nodes in a data structure. NOTE: This is not necessarily a binary search tree.

